

SISTEMAS OPERACIONAIS • ATIVIDADE PRÁTICA EM GRUPO



Seja o gerente de memória

Referência: slides 1 a 22: endereços, Base e Limite, partições, fragmentação, First/Next/Best-Fit e Buddy System

Grupo	Integrantes

Como a atividade funciona

Nesta aula o grupo vai construir, do zero, um pequeno simulador de alocação de memória e usá-lo para comparar estratégias. O simulador cresce em três etapas. Em cada etapa, o grupo primeiro prevê o resultado à mão e só depois executa o programa. Quando a previsão e o programa discordarem, descubram qual dos dois está errado: é aí que está o aprendizado.

- Grupos de 2 a 4 alunos, um computador por grupo.
- Linguagem livre. Não é preciso interface gráfica: imprimir o estado da memória no terminal é suficiente.
- Todos os tamanhos e endereços estão em KB. A memória disponível para processos tem 1.024 KB, com endereços de 0 a 1.023 (o SO fica fora dessa área).
- Anotem previsões e resultados neste roteiro. No fechamento, o relator leva os números do grupo ao quadro.

Papéis (troquem a cada etapa)

Papel	Responsabilidade
Piloto	Digita o código. Só ele usa o teclado.
Navegador	Dita a lógica, revisa o código enquanto é escrito e aponta casos de borda.
Verificador	Resolve a etapa à mão, no papel, sem olhar a tela, e confronta com o programa.
Relator	Registra resultados e dúvidas neste roteiro e apresenta os números no fechamento.

Grupos de 2 ou 3: acumulem papéis (por exemplo, piloto + relator e navegador + verificador).

Etapa	Tema	Tempo
1	Base e Limite: relocação e proteção	15 min
2	Partições fixas e dinâmicas (First, Next e Best-Fit)	35 min
3	Buddy System	30 min
4	Fechamento coletivo	15 min

Etapa 1 – Base e Limite (15 min)

Implementem uma função que simula o que a MMU faz com os registradores Base e Limite (slides 8 a 10):

```
traduz(base, limite, relativo)
→ endereço físico, se  $0 \leq \text{relativo} < \text{limite}$ 
→ "TRAP", caso contrário
```

Considerem dois processos: P_A com BASE = 20.000 e LIMITE = 6.000, e P_B com BASE = 30.000 e LIMITE = 2.500. O verificador preenche a coluna de previsão antes de o programa rodar.

Processo	Endereço relativo	Previsão (à mão)	Resultado do programa
P_A	0		
P_A	4.096		
P_A	5.999		
P_A	6.000		
P_B	2.400		
P_B	2.500		

Para discutir no grupo

1. Uma compactação move P_A, e o SO passa a usar BASE = 12.000. O que precisa mudar no código de P_A? Para qual endereço físico vai o relativo 4.096 agora?
2. Em um sistema real, quem executa essa verificação a cada acesso: o SO ou o hardware? Por quê? E quando o SO entra em ação?
3. Por que a condição é "relativo < LIMITE" e não "relativo ≤ LIMITE"?

Etapa 2 – Partições fixas e dinâmicas (35 min)

Esta sequência de eventos (trace) será usada nas etapas 2 e 3:

#	Evento	Processo	Tamanho (KB)
1	Aloca	P1	260
2	Aloca	P2	160
3	Aloca	P3	120
4	Aloca	P4	180
5	Aloca	P5	130
6	Libera	P4	–
7	Libera	P1	–
8	Libera	P2	–
9	Aloca	P6	170
10	Aloca	P7	180
11	Aloca	P8	200
12	Aloca	P9	200

Regras válidas para todas as estratégias

Não há fila de espera: se um pedido não puder ser atendido no momento, o processo é **negado** e descartado. Se depois aparecer a liberação de um processo negado, ela é ignorada.

Na alocação dinâmica, o processo ocupa o **início** do buraco escolhido, e a sobra continua como buraco. Ao liberar, buracos adjacentes são fundidos.

2a. Partições fixas (à mão, sem programar)

A memória foi dividida no boot em 4 partições, nesta ordem de endereço: 0 = 128 KB, 1 = 128 KB, 2 = 256 KB e 3 = 512 KB. Há fila única, e cada processo vai para a menor partição livre em que cabe.

Enquanto o piloto começa a 2b, o verificador preenche esta tabela:

#	Evento	Partição (0–3) ou NEGADO	Frag. interna total após o evento (KB)
1	Aloca P1 (260)		
2	Aloca P2 (160)		
3	Aloca P3 (120)		
4	Aloca P4 (180)		
5	Aloca P5 (130)		
6	Libera P4		
7	Libera P1		
8	Libera P2		
9	Aloca P6 (170)		
10	Aloca P7 (180)		
11	Aloca P8 (200)		
12	Aloca P9 (200)		

2b. Partições dinâmicas (programar)

Representem a memória como uma lista de buracos (início, tamanho) ordenada por endereço e implementem as três políticas do slide 18:

- **First-Fit:** primeiro buraco suficiente a partir do endereço 0.
- **Next-Fit:** guarde um ponteiro com o fim do último bloco alocado (inicialmente 0). A busca começa no buraco que contém o ponteiro ou no primeiro buraco depois dele e, se chegar ao fim da memória, recomeça do início. Liberações não movem o ponteiro.
- **Best-Fit:** menor buraco suficiente; em caso de empate, o de menor endereço.

Previsão antes de rodar

Desenhem a memória após o evento 8. Por que esse estado é o mesmo para as três políticas? Em seguida, prevejam onde P6 e P7 serão colocados em cada política:

Política	P6 (170) começa em	P7 (180) começa em	O programa confirmou?
First-Fit			
Next-Fit			
Best-Fit			

Resultados do programa (trace completo)

Política	Pedidos negados	Havia livre total suficiente?	Livre total ao final	Maior buraco ao final	Nº de buracos
First-Fit					
Next-Fit					
Best-Fit					

Para discutir no grupo

1. Algum pedido foi negado mesmo havendo memória livre total suficiente? Que tipo de fragmentação é essa?
2. Se o SO compactasse a memória imediatamente antes de cada negação, quais pedidos passariam a ser atendidos? Qual é o custo dessa compactação (slide 17)?
3. Nas partições fixas, P4 (180 KB) foi atendido? O que isso mostra sobre o número e o tamanho das partições?
4. A política vencedora neste trace é sempre a melhor? Guardem essa pergunta para o desafio extra.

Etapa 3 – Buddy System (30 min)

Implementem um alocador Buddy para os mesmos 1.024 KB (slides 20 a 22), seguindo estas regras:

- O pedido é arredondado para a menor potência de 2 suficiente. O bloco mínimo é de 16 KB.
- Use o menor bloco livre de tamanho suficiente e, entre blocos do mesmo tamanho, o de menor endereço. Se for preciso dividir, o processo fica com a metade esquerda e a metade direita vira bloco livre.
- Ao liberar, o bloco só se funde com o seu buddy (o irmão gerado na mesma divisão), e somente se ele estiver inteiramente livre. A fusão se repete enquanto for possível.

Dica: com endereços contados a partir do início da memória, o buddy de um bloco de tamanho t no endereço e está no endereço $e \oplus t$. Uma estrutura simples é um dicionário tamanho $\rightarrow$ lista de endereços livres.

3a. Validação com a sequência do slide 22

Rodem a sequência abaixo e confirmem com a previsão do verificador. O slide deixa duas fusões em aberto ("não funde se...", "funde somente se..."): agora vocês vão descobrir o que realmente acontece.

Operação	Bloco (KB)	Endereço inicial	Houve fusão? Qual bloco livre resultou?
request A = 70 K			
request B = 35 K			
request C = 80 K			
free A			
request D = 60 K			
free B			

Desenhem o mapa final da memória (0 a 1.024 KB), marcando cada bloco como livre ou ocupado:

3b. O trace da etapa 2 no Buddy

Rodem o mesmo trace dos eventos 1 a 12 e completem a comparação final:

Estratégia	Pedidos negados	Tipo de fragmentação dominante	Fragmentação interna máxima (KB)
Partições fixas			
First-Fit			–
Next-Fit			–
Best-Fit			–

Estratégia	Pedidos negados	Tipo de fragmentação dominante	Fragmentação interna máxima (KB)
Buddy			

Para discutir no grupo

1. No mapa final da validação existem blocos livres vizinhos que não se fundiram. Por que isso não é um erro do alocador?
2. No trace, por que P4 (180 KB) é negado no Buddy logo no evento 4, se o total pedido até ali é bem menor que 1.024 KB?
3. O Buddy troca que tipo de fragmentação por qual? Em que situação essa troca compensa?

Etapa 4 – Fechamento (15 min)

O relator leva ao quadro a tabela da etapa 3b. A turma compara os números entre os grupos: resultados diferentes costumam indicar uma regra implementada de forma diferente, e vale descobrir qual.

Desafio extra (para grupos que terminarem antes)

Construam um trace curto, de no máximo 8 eventos, em que o **Best-Fit nega mais pedidos que o First-Fit**. Troquem o trace com outro grupo, que deve verificá-lo no próprio simulador.

Variação: um trace em que o Buddy nega um pedido que a alocação dinâmica (qualquer política) atenderia.